

Doubly Linked List (All Questions)

(Question # 01)

```
void sortedinsertion()
```

```
{ if(head == NULL) // if no node before  
  head = tail = newnode; }
```

```
{ if(val <= h->data) // if val is < head  
  // insert at beginning  
  ROC; }
```

```
{ if(val >= tail->d) // if val > tail  
  // insert at end  
  ROC; }
```

```
Node * temp = head; // for traversal
```

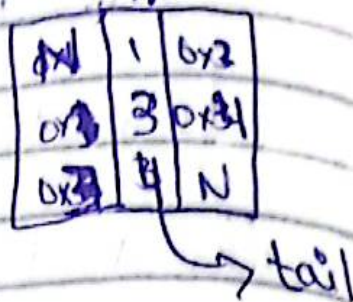
```
while(temp != NULL && temp->data < val) // if, then run  
{  
  temp = temp->next; // iterate temp  
}
```

```
newnode->next = temp;  
newnode->prev = temp->prev;  
temp->prev->next = newnode;  
temp->prev = newnode;
```

~~managing nodes linking~~
Done Before

(DRY RUN) ... head

Suppose: Our list is:
value to be entered is 2



1) if (head == NULL) X

2) if (val <= head->data) X $2 <= 1$ X

3) if (val >= tail->data) X $2 >= 4$ X

temp = 0x1 (1)

4) while (t != N && t->data < val) ✓
{
 t = t->next; t = 0x3
}

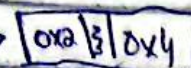
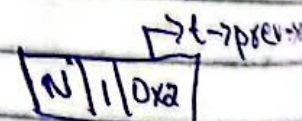
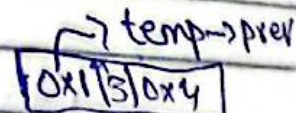
5) while (—) X t->data != val

newnode->next = temp;

newnode->prev = temp->prev;

temp->prev->next = newnode;

temp->prev = newnode;



So,

0x1	1	0x2
0x1	2	0x3
0x2	3	0x4
0x3	4	N

newList
(sorted)

Question # 02

```
void deleteOccurrences()
```

```
{  
    if (head == NULL) { return; } // list empty
```

```
    Node* temp = head;
```

```
    int count = 0;
```

```
    // for traversal
```

```
    // for counting occurrences
```

```
    while (temp != NULL)
```

```
{
```

```
        if (temp->data == val)
```

```
            // if val found
```

```
{
```

```
            Node* toDelete = temp;
```

```
            if (temp == head)
```

```
                // if at head
```

```
{
```

```
                // managing head,
```

```
{
```

```
            else if (temp == tail)
```

```
                // if at tail
```

```
{
```

```
                // managing tail;
```

```
{
```

```
            else
```

```
                // if b/w somewhere
```

```
{
```

```
                temp->prev->next = temp->next;
```

```
                temp->next->prev = temp->prev
```

```
                temp = temp->next;
```

```
// Deleting  
// linking prev &  
// next of temp
```

```
{
```

```
            delete toDelete;
```

```
            count++;
```

```
            // increment c
```

```
        }
```

```
        else { temp = temp->next; }
```

```
            // increment temp
```

```
}
```


Suppose: Our list is

delvalue to del 2

temp = 0x1 (1) (head)

0x1	N	1	0x2
0x2	0x1	2	0x3
0x3	0x2	2	0x4
0x4	0x3	3	0x5
0x5	0x4	3	N

•) while (t != NULL) ✓

if (t->data == zval) X
else
{

t = t->next;

t = 0x2

}

•) while (t != N) ✓

if (t->d == val) ✓
{

Node* todel = t;

todel = 0x2

if (t == head) X

else if (t == tail) X

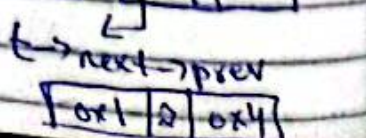
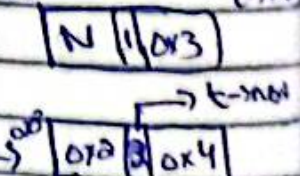
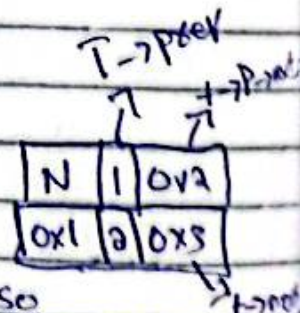
else
{

t->prev->next = t->next;

t->next->prev = t->prev;

t = t->next; (t = 0x3)

}



delete todel;

c++; c=1

0x1	0x1	1	0x3
0x3	0x1	2	0x4
0x4	0x3	3	0x5
0x5	0x4	3	N

again while(-)✓

if(-)✓

{

todel = t;

todel = 0x3

if(t = h) x

else if(t == tail) x

else

{

t->p->n = t->next; →

t->n->p = t->prev;

t = t->next; (t = 0x4)

}

delete todel;

c++; c=2

Before

0x1	0x1	1	0x3
0x3	0x1	2	0x4
	0x4	3	0x5

after

	N	1	0x4
	0x1	3	0x5

again: while(-)✓

if(-) x

else

t = t->n;

t = 0x5

Final List

again while(-)✓

if(-) x

else

t = t->n;

t = N

again while(-) x

t = N

Question # 03

```
void sum()
{
    if (—) { return; } // 0 or 1 node
```

```
Node* left = head;
Node* right = tail;
```

```
int count = 0;
```

```
while (L != R && L->prev != R)
{
```

```
    int CS = L->data + R->data;
```

```
    if (CS == S)
    {
```

```
        (prompt) count++;
```

```
        left = L->next;
```

```
        R = R->prev;
```

```
    }
```

```
    else if (CS < S)
```

```
    {
```

```
        L = L->next;
```

```
    }
```

```
    else
```

```
    {
```

```
        R = R->prev;
```

```
    }
```

DRV RUN

Suppose: value to find sum = 17
be ~~deleted~~

left = head L = 0x1
 Right = tail R = 0x7
 int count = 0 C = 0

0x1	N	1	0x2
0x2	0x1	2	0x3
0x3	0x2	3	0x4
0x4	0x3	4	0x5
0x5	0x4	5	0x6
0x6	0x5	6	0x7
0x7	0x6	7	N

while(—) ✓

int CS = L → d + R → d; (CS = 7+8)

if (CS = S) X 8 ≠ 7
 else if (CS < S) X 8 < 7
 else
 { R = R → pnext; } R = 0x6

while(—) ✓

int CS = L → d + R → d; (CS = 7+7)

if (CS = S) ✓ 7 = 7

prompt;
 count++;
 L = L → next; L = 0x2
 R = R → pnext; R = 0x5
 }
 }

while(—) ✓

in CS = L → d + R → d; (2+5=7)

if(—) ✓
 { prompt;
 C++;
 L = L → next; L = 0x3
 R = R → pnext; R = 0x4
 }

again: while(—) ✓

{
int CS = L → R → d; (3+4=7)

if(—) ✓

Printf

ctt;

L = L → next;

0x4

R = R → prev;

0x3

} ...

again: while(—) ✗ (left → prev = right)

So, pairs will be: (1, 6), (2, 5), (3, 4)

Question # 04

void rotate()

{

if(—) { return }

// 0 or 1 node

length = getlength();

// found total nodes

n = n % length;

// to get n in range

if(n > 0)

// no need to rotate

if(n < 0)

// opposite rotate

-5 → 0

-4 → 1

-3 → 2

-2 → 3

-1 → 4

ROC

Detail in (DRY RUN)

Question # 05

```
void merge(L1 &L2)
```

```
{  
    Node* h1 = head1;
```

```
    Node* h2 = L2->getHead();
```

// storing both heads

```
    if (h1 == NULL)
```

// if L1 is empty

```
    {  
        head = h2;
```

```
        updateTail();
```

```
        return;
```

```
    }
```

```
    if (h2 == NULL)
```

// if L2 is empty

```
    {
```

```
        return;
```

```
    }
```

```
    Node* new new head = NULL;
```

```
    Node* new new tail = NULL;
```

// for merged list

```
    if (h1->d <= h2->d)
```

```
    {  
        new new head = h1;
```

```
        h1 = h1->next;
```

```
    }
```

// if yes, store head & iterate

```
    else
```

```
    {  
        new head = h2;
```

```
        h2 = h2->next;
```

```
    }
```

// if yes, store head & iterate

```
    new tail = new head;
```

// same;

```
    new head->prev = NULL; // update prev
```

```
    }
```

RDC explained in DRY RUN

Question # 06

void Displaytree()
// later in tree

void Inorder()
// later in tree
same just prompt replaced by
insertion

void convert()
// calling inorder

Question # 07

if(head == null) // 0, 1, or 2 nodes

bool found = false;

Node* first = head;

while (first != null)

{

explained in (DRY_RUN)

}

if (found == false) // not triplets
{ return; }

DRY RUN

Suppose: $n = 5$

Our list is:

first = head (0x1)

Now while (f ≠ N) ✓

Node* S = F → next;

Node* T = tail;

S = 0x2

T = 0x4

0x1	N	1	0x2
0x2	0x1	2	0x3
0x3	0x2	3	0x4
0x4	0x3	4	0x5
0x5	0x4	5	0x6

while (—) ✓

int sum = F → d + S → d + T → d; (1 + 2 + 4 = 7)

if (sum ≠ n) ✗ 7 ≠ 5

else if (sum < n) ✗ 7 < 5

else

{

third = T → prev; T = 0x3

again while (—) ✓

int sum = F → d + S → d + T → d; (1 + 2 + 3 = 6)

if (sum = n) ✓

{

prompt, true!

S = S → next;

S = 0x3

T = T → prev;

T = 0x2

again while (—) ✗ (T → next = S)

! and so on
again after iterating first
inner loop

Question # 09

```
void reverse()
```

```
{
```

```
    Same as of SLL;
```

```
}
```

```
void reversehelper(Node* head, int k)
```

```
{
```

```
    //
```

// Explained in DRY_RUN, remaining same as SLL just new is prev.

(DRY_RUN)

Suppose: Our list is :

k=2

0x1	N	1	0x2
0x2	0x1	2	0x3
0x3	0x2	3	0x4
0x4	0x3	4	NULL

prevnode = Null;

curr = 0x1

NN = Null;

C = 0

o) while(—) ✓

NN = curr->next;

curr->next = prevnode; → [N | NULL]

if (NN != NULL) ✗

prevnode = curr

curr = NN;

C++;

NN = 0x1

curr = 0x2

C = 1

if (NN != NULL) {

head->next = reversehelper(curr->next, k);

~~Node~~
// this will be called recursively 

Node PN = Null;

curr = head

NN = Null;

int c = 0;

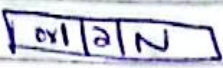
→ It is 0x2 now

while(→) ✓
{

NN = C → next;

NN → 0x3

C → next = PN;



if (PN != N) ✗

prevN = curr

curr = NN;

PN = 0x2

curr = 0x3

if (NN != NULL)
{

h → next = recu(C →);



!
and so on

remaining part is just same as
SL reversing in groups.

just a new check on prev^{*}

Question # 10

```
void split(CDL &sh, CDLL &sh)
```

```
{  
    if (head == NULL) // if no node
```

```
        Node * slow = head;  
        Node * fast = head;
```

```
    while (1)
```

```
    {  
        fast & slow; // same, finding mid  
    }
```

```
    Node* h1 = head;
```

```
    Node* h2 = slow->next; // separating head
```

```
    tail = head->prev;
```

```
    // for saving prev
```

```
    slow->next = h1;
```

```
    // separating prev  
    & making CDLL
```

```
    h1->prev = slow;
```

```
    // First half separate  
    & make CDLL
```

```
    tail->next = h2;
```

```
    h2->prev = tail;
```

```
    // updating h2's prev
```

```
    fh = head = h1;
```

```
    sh = head = h2;
```

```
    // Assigning sep  
    heads
```

(DRV_RUN)

Suppose our list is

slow = 0x73
fast = 0x75

h1 = 0x1
h2 = 0x74

tail = 0x6

1	0x1	0x6	1	0x2
2	0x2	0x1	2	0x3
3	0x3	0x2	3	0x4
4	0x4	0x3	4	0x5
5	0x5	0x4	5	0x6
6	0x6	0x5	6	0x1

slow → next = h1

0x2 → next = 0x1

h1 → prev = slow;

0x2	2	0x1
-----	---	-----

0x3	1	0x2
0x4	2	0x3
0x2	3	0x1

tail → next = h2;

h2 → prev = ^{tail}tail;

0x5	6	0x4
-----	---	-----

0x6	4	0x5
0x4	5	0x6
0x5	6	0x4

fh.head = h1;

sh.head = h2;

// Two list created
by separated